PRE-BUILD TECHNICAL REVIEW · 2026-08-07

Implementation Plan — building the ~1.0T two-stage corpus

Written 2026-08-07. Companion to `docs/FINAL-DATASET-REPORT.md`, which decides *what* the corpus contains. This document decides *how it gets built*, and it is written to be read before any job is submitted.

Every claim here is graded. `MEASURED-IN-CODE` means a file and line in this repository says it. `MEASURED` means a real run or a real byte read produced the number. `DERIVED` means arithmetic from a measured input, with the inputs named. `CARD` means an upstream dataset card asserts it and nobody has checked. `UNVERIFIED` means plausible and unchecked — flagged, never used to justify a decision.

---

0. The executive summary, and the five things that would have gone wrong

The pipeline that will build this corpus already exists and already ran: 27 bundles, 10,049 shards, **251.2B tokens** published on 2026-08-05. Scaling it to 1.0T is mostly arithmetic — but five defects surfaced in this review, and **not one of them fails loudly.** Each either discards work silently or produces a corpus that passes every gate while being wrong.

#	blocker	consequence if unfixed	fix
1	**Ordinals shift when a source is added**	adding one 4B source renames **98% of shards** and voids **882B tokens**	freeze the full plan first — **0 code**
2	**The FinePhrase de-dup predicate is never called**	59.8B declared synthetic is **~18.5B distinct**; the epoch guard reports green at ~4× true exposure	~5 lines, at the reader
3	**The dedup set OOMs at 1T**	19.37 GB for one bundle in a 15.03 GB container	flat `np.uint64` → 1.80 GB
4	**The decontamination index is built from 5-shot renders**	149,777 exact hashes are **dead** for MMLU/ARC/HellaSwag	rebuild from raw fields, one CPU job
5	**The reader budgets 18 TB to fetch 4.21 TB**	4.28× over-read, half of it from the val split alone	one constant + ~30 lines

Two more that are not blockers but would embarrass us: `tokenizers` **is not a declared dependency**, so production resolves whatever PyPI served that morning — and every Cosmopedia document begins with a leading space, which changes its first token under byte-level BPE.

The two most instructive are below; the rest are in their own sections.

The first is an ordering trap. The plan of record says to "ingest source by source, each as a separately-approved platform job." Executed literally, that discards nearly everything each time. A shard's ordinal comes from a single per-split counter walking the plan in alphabetical order (`corpus.py:352-359`, MEASURED-IN-CODE), so inserting a source shifts every source that sorts after it. Simulated on the real stage-1 mix: **adding one 4B-token source renames 35,278 of 35,998 shards — 98% — and voids 882B tokens**, about 23 hours of tokenize. The shard path carries the ordinal and the path is inside `manifest_sha256`, so a rename is a different dataset identity, and `bundle_is_done` rejects every prior receipt.

The fix costs no code: compute the complete plan for every source of both stages **before the first job**, then run bundles in whatever order approvals arrive. `--shard/--of` already slices bundles and `bundle_is_done` already skips finished ones. The consequence is real though — **the mix has to be final before the first token is written.** Freezing the mix, not starting the ingest, is the blocking step.

The second is a correctness gap with a green light on top of it. The four FinePhrase configs are one corpus rephrased four ways over the same ~339M FineWeb-Edu documents, **MEASURED at 91.0–92.9% pairwise id overlap**. The code to de-duplicate them exists, is tested, and is verified balanced to within 0.27 percentage points — and **production never calls it.** `keeps_id`'s three call sites are a reporting function, a test, and a measurement script. `IdSet.contains`, the anti-join primitive, has zero callers anywhere in the repository.

So of 59.8B declared synthetic tokens, roughly **18.5B are distinct source documents** (DERIVED from the measured overlap) and ~41B are rephrasings of documents already present under a different format label. No deduplication method catches this, at any scope, because four rephrasings of one document are four different strings. And the epoch guard reports deep green while it happens: `epochs_for` divides by the *declared* pool size, so a mixture drawing 0.25 from each of four synthetic sources reads 0.33–0.50 epochs on the dashboard while true per-document exposure is about `4×`.

A passing verification artifact for a code path production does not execute is the most dangerous shape a gap can take, because every audit looks green.

Both fixes must land before the plan is frozen, and the FinePhrase one must land before anything is tokenized: the registry's own trap text says that after tokenization there is no document→id mapping and it cannot be retrofitted.

---

1. What already works, so it does not get rebuilt

Worth stating plainly, because the temptation when a review finds two blockers is to distrust everything around them.

property	evidence
The plan is a pure function, content-addressed	`plan_document` takes no clock, no S3, no environment; `plan_id` is the sha256 of its own JSON. Two runs that agree on the plan agree on every key they will write. MEASURED-IN-CODE, `corpus_build.py:182–289`
The build is deterministic	Nine bundles re-run under `--force` reproduced **byte-identical digests**. MEASURED, `artifacts/reservoir/realized-tokens.json`
Resume is safe and honest	`bundle_is_done` re-HEADs every shard and compares **sizes**, not mere presence. MEASURED-IN-CODE, `corpus_build.py:387–427`
The val split cannot leak	`is_held_out` is a pure hash of `(source, doc_id)` with no PRNG, so both splits are decided from one read. MEASURED-IN-CODE, `corpus.py:368–402`
Memory is bounded during pack	The sink uploads each shard and drops it, so at most one ~100 MB buffer is resident. MEASURED-IN-CODE, `corpus_build.py:461–468`
The double-encode trap is already closed	`min_tokens` filters from the ids `encode_batch` already produced. The alternative cost 91% of build compute on 1 of 32 cores. MEASURED, `corpus_pack.py:230–250`
Ordinal reuse is not silently accepted	`allocate_ordinals` raises when a plan names one stream twice. MEASURED-IN-CODE, `corpus.py:340–348`

The determinism result is what makes the coarse resume unit survivable: a lost bundle can be re-run and will produce the same bytes.

---

2. The ordering constraints — why the steps cannot be reordered

Several of these are one-way doors. The build is a straight line, and each step below names what breaks if it moves.

#	step	why it is pinned here
0	**Stage sources to S3**	Optional but recommended (§3). Everything downstream reads in-region afterwards
1	**Freeze the complete plan**	Adding a source later renames 98% of shards (§0)
2	**Apply the FinePhrase id partition**	After tokenization there is **no document→id mapping**; it cannot be retrofitted
3	**Read + carve val**	`is_held_out` is per-document, so a val bundle must read the train documents beside it
4	**Dedup, then decontaminate**	Dedup is one sha256; decontamination is up to `len(words)-12` blake2b hashes. Cheap predicate first means a duplicate is never contamination-checked
5	**Neutralize boundary markers**	Must precede the encode. Afterwards the marker is an id indistinguishable from the boundary we append
6	**Tokenize, appending EOS**	The `min_tokens` floor applies here, from ids already computed — never a second encode
7	**Pack + upload**	One shard resident at a time
8	**Receipt**	Per bundle. This is the resume unit
9	**Publish to landing**	⚠️ A `manifest.json` fires EventBridge → **automatic promotion**. There is no publish-without-promote mode
10	**Gate A validation**	Recomputes from bytes and rejects on mismatch

Step 4's ordering is not a style preference: contamination checking is the expensive half, and duplicates are common in web crawls, so testing the cheap predicate first is a real saving.

Step 9 deserves emphasis because it is the one irreversible step. Writing a manifest **is** publishing. To stage without promoting, the validator job must be cancelled or the EventBridge rule disabled first.

---

3. Proposed change: stage the sources to S3 once

The problem. `_reader_for` streams live from HuggingFace inside the Batch container, and its own docstring flags this as the least-tested path in the build: "⚠️ *UNVERIFIED against live HF from inside a Batch container — every offline test injects* `documents=` *instead.*" MEASURED-IN-CODE, `corpus_build.py:900-906`.

The proposal. A pass 0 that copies raw source files from HuggingFace to `s3://edullm-landing/_src/<source>/`. No parsing, no filtering, no tokenizing. Every later pass reads in-region.

3.1 ⚠️ First, the number that dominates everything: the pipeline reads 18 TB, not 4.2 TB

This is the largest cost in the build and it is an artifact of two constants, not of the data.

`_reader_for` sets `budget = int(bundle.tokens * _CHARS_PER_TOKEN * _FILTER_HEADROOM / keep_rate)` with `_CHARS_PER_TOKEN = 6.0` and `_FILTER_HEADROOM = 1.5` (`corpus_build.py:865,881`).

	figure
real text needed for 1.0T tokens, at a mix-weighted **0.2374 tok/byte** (19 measured values)	**4.21 TB**
char budget per token	6.0 × 1.5 = **9.0**, against a measured mean of **4.31 B/token → 2.09× over-read**
train split budgeted read	9.00 TB
val split budgeted read (`keep_rate = VAL_FRACTION = 0.005`, so the budget is divided by 0.005)	**9.00 TB**
total budgeted read	**18.00 TB — a 4.28× amplification**

The val split alone roughly doubles the download, and it is not a bug: `is_held_out` is a hash of the document id, so a val document cannot be located without reading the train documents beside it. But budgeting `tokens × 9 / 0.005` for a 5B-token val split is the wrong instrument. **Two fixes, both cheap:**

1. **Correct** `_CHARS_PER_TOKEN` from 6.0 to ~4.6 (measured 4.31 plus real headroom). 18 TB → ~9.2 TB for a one-line change.
2. **Give val bundles file-sharding instead of a 200× budget divisor.** `_shard_slice` already exists and is already imported — roughly 30 lines, and the highest-value change in the whole audit.

3.2 Then stage the sources once

	figure	grade
stage once	**4.21 TB** of real source text (already compressed as parquet / jsonl.gz)	DERIVED from 19 measured tok/byte values
S3 Standard storage	**~$97/month**, so ~**$194** for a two-month build window	DERIVED at $0.023/GB-month
HF → AWS transfer	**free** (AWS does not charge ingress)	—
re-read 18 TB, **staged in-region**	**0.20–4.0 h** (0.5 h at 8 × 10 Gbit/s)	DERIVED
re-read 18 TB, **live from HF**	**8–40 h** at 5–1 Gbit/s, and rate-limit exposed	DERIVED

Staging is what makes the amplification affordable rather than fatal: the same 18 TB costs in-region bandwidth instead of internet bandwidth. Fix the constants *and* stage, and the read stops being the binding constraint at all.

Four benefits beyond speed, the first of which makes §5's dedup design possible:

1. **Extra passes become cheap** — a second read costs ~0.5 h instead of 8–40 h.
2. **Resume never re-hits HuggingFace.** No rate limits, no 429s, no revision drift mid-build.
3. **Reproducibility.** An HF revision can move under us; staged bytes cannot.

4. **It retires the untested path.** The live-HF read stops being a production dependency.

Honest cost: ~$194 and one extra job. Recommended.

3.3 For contrast, the tokenize costs about $40

At the measured 10.5 M tok/s across 32 vCPU and `c7i` on-demand pricing (~$0.04462/vCPU-hour, UNVERIFIED against a live price API), 1.0T tokens is **~$38** — and it is $38 at 32, 64, or 96 vCPU, because the work is fixed and the rate scales. Roughly $79 with the 2.09× over-read included.

So tokenization is neither the wall-clock bottleneck nor a cost item. That is the context for §7's gigatoken recommendation.

And there is no Batch timeout to design around. `INGEST-CALIBRATION.md:19-22` corrects this in-repo: the 7200 s figure was *our own setting*, not a platform ceiling. The real constraint is the **128 vCPU compute environment**.

---

4. Sources — the format and encoding traps

Each of these silently produces a wrong corpus rather than an error, which is why they are listed individually.

4.1 DCLM cannot be drawn from the sample, and the full corpus is unreadable

The plan wants **378B tokens**. Three repositories exist and they differ in format:

repo	format	tokens	usable
`HuggingFaceFW/dclm_100BT`	parquet	**114.7B MEASURED**	✅ but only 30% of what is needed
`mlfoundations/dclm-baseline-1.0`	`.jsonl.zst`	~3,764B DERIVED	❌ **unreadable**
`mlfoundations/dclm-baseline-1.0-parquet`	parquet, 27,938 shards	~3,764B DERIVED	✅ **use this**

VERIFIED IN CODE: `READABLE_FORMATS = frozenset({"parquet", "json.gz"})` (`corpus_build.py:127`), and `_assert_readable` rejects anything else **at plan time** (`:171`). `corpus_read.py:774-775` states it directly: "`.zst` *is NOT among them … needs a zstandard dependency this package does not declare.*" `pyproject.toml` declares only `boto3` and `numpy<2.5`.

Two traps in the parquet mirror:

- Its row count is an **estimate**. `/statistics` returns a permanent HTTP 501, and `/size` returns `num_rows=779,982` with `partial:true` — the converted head, **0.03% of the corpus**. A pipeline reading `num_rows` from `/size` sizes this source at **1/3800 of reality**.
- It has **6 columns to the original's 8**, dropping the WARC metadata struct and `warcinfo`. The card calls it "an identical copy"; it is identical in `text`, not in provenance.

4.2 FinePhrase — the nested column, and the trap that does not raise

The rewrite lives at `path_in_schema` `rollout_results.list.element.text`. The top-level `text` column holds the **original FineWeb-Edu document** — its `dataset` field literally reads `HuggingFaceFW/fineweb-edu`.

A flat leaf scan finds `text` **twice**, and `.names.index("text")` returns the original. That builds a corpus of unrephrased web text labelled synthetic, and **no hash, size, or decode check catches it**: the bytes are real text, correctly tokenized, in valid ids.

Worse, the near-miss spelling `rollout_results.text` **does not raise either** — it returns a table with zero columns. Only the exact `path_in_schema` works. Verified through the finished reader; both near-miss spellings are rejected.

4.3 FineWeb-Edu's overlap with FinePhrase is total, not partial

`sample-100BT ⊂ sample-350BT`, and `sample-350BT` is FinePhrase's exact parent. So **100%** of an edu-web draw has a synthetic sibling. The free fix: draw synthetic from the ~242M `sample-350BT` ids **not** in `sample-100BT`.

4.4 Two sources ship a domain, and cardinality is permanent

`stackv2-edu` carries 73 languages via `metadata.gha_language`; `stackexchange` carries ~180 sites via `metadata.site`. **Every distinct value becomes a directory inside** `manifest_sha256`**.** Fold to the top ~20 with the rest as `other`. The fold map is a **reader argument, not a spec field**, because a streaming pass cannot know the top 20 before it has read everything.

4.5 Short documents are a publish blocker, not a preference

`families/pretrain.json` sets `eos_fraction_max: 0.05`, which is a **mean-document floor of 20 tokens**. FinePhrase has a sampled rewrite that is **12 tokens** long. A mean under 20 puts the packed shard's EOS fraction over the bound and **Gate A rejects it — after the tokenize and the upload.** `corpus.MIN_DOC_TOKENS = 64` is what prevents that.

Measured means, for reference — all clear the bound comfortably:

source	tokens/doc	EOS fraction
pubmed	7,917.9	0.000126
peS2o	6,474.0	0.000154
finewiki	1,320.1	0.000758
stackexchange	728.8	0.001372
whole reservoir	**814.9**	**0.001227**

Note `ubuntu-irc` measures 8,650 tokens/doc — its IRC turns are concatenated per document, so it is *not* the short-document risk. FinePhrase is.

---

5. Deduplication

5.1 What the code does today

`dedup_and_decontaminate` drops exact duplicates then contaminated documents, keying on the sha256 of NFC-normalized, right-stripped text. `SeenHashes` stores the **top 128 bits as an int**, not the hex string — a deliberate memory fix, MEASURED with `tracemalloc`: 85.9 B/entry against 154.9 B/entry for `set[str]`.

Scope is per-bundle. `corpus_build.py:482` passes no `seen=`, so `corpus_filter.py:302` allocates a fresh set per bundle. That was a documented choice at 251B — "dedup here is within a bundle, which is where duplicates actually cluster."

5.2 Why it does not scale unchanged

Document count is **MEASURED**, not estimated: 308,291,107 documents for 251,218,001,920 tokens = **814.9 tokens/document** (`artifacts/reservoir/realized-tokens.json`, 27 receipts). At 1.0T that is **1.23B documents**, with 2.0B as a planning bound if the mix shifts toward synthetic (FinePhrase averages 263–442 tok/doc against pubmed's 7,918 — a 30× spread).

⚠️ **And the current design does not merely miss cross-source duplicates — it runs out of memory.** This is the third blocker, and it appears in no design document.

The Batch container is **14 GiB (15.03 GB)**, sized from a measured worst-bundle resident of ~12.1 GB. The largest bundle *by document count* at 1T is not the largest by tokens: it is `synthetic-finephrase-table--train`, whose mean document is only 263 tokens — 56,839,223 documents at 252B becomes **225.6M at 1T**.

representation	worst bundle @1T	global @1T	fits 15.03 GB?
`set[int]` 128-bit (**current code**)	**19.37 GB**	105.4 GB	**NO**
`set[int]` 64-bit	17.57 GB	95.6 GB	**NO**
flat `np.uint64`, 16 B/key	3.61 GB	19.6 GB	yes
flat `np.uint64`, 8 B/key	**1.80 GB**	**9.8 GB**	**yes**

Dedup is only one resident structure: add ~0.45 GB for the decontamination index, 0.4 GB tokenizer, 0.5 GB pyarrow row group, plus the interpreter. **Three of the four FinePhrase bundles and `stackv2-edu` all exceed the container on the dedup set alone — with no global dedup attempted.**

VERIFIED MYSELF, and it corrects a natural intuition: narrowing the key *inside* a Python `set` does not help. `sys.getsizeof` gives 44 B for a 128-bit int and 36 B for a 64-bit one, so against the codebase's measured 85.9 B/entry the set's own slot overhead is **41.9 B/entry and width-independent** (CPython stores a pointer plus a cached hash, not the value). Narrowing 128→64 buys **9.3%, not 50%**.

The 5–11× win comes entirely from **leaving the `set` for a flat numpy array**.

5.3 Proposed: a sort-based dedup pre-pass, partitioned 256 ways

Two independent constraints point at the same design, which is what makes it the right one rather than merely the cheapest.

Constraint 1 — memory. A flat `np.uint64` accumulator plus `np.unique` is 8 B/key: **1.80 GB for the worst bundle**, 11× denser than the current set. But a flat array cannot answer "seen?" incrementally, so it forces a separate pass.

Constraint 2 — determinism. A shared mutable filter threaded through the build would make the output depend on bundle execution order, destroying the byte-identical-rerun property that §1 shows is already verified. A pre-pass that emits an immutable keep-list preserves it.

The design. Pass 1 reads the staged text and emits `(hash, source, ref)` triples. The hash space is partitioned 256 ways; each worker owns partition `p` and therefore sees documents from **every** source whose hash falls in `p`. Cross-source duplicates land in the same partition by construction, so the result is **exact global dedup with zero shared state** — no Bloom false positives discarding real documents, no coordination. Sort, unique, resolve winners by an explicit source priority (§5.5), emit a keep-list. Pass 2 builds shards against it.

partitions	resident per worker @1.23B docs
64	1.65 GB
128	0.82 GB
256	**0.41 GB**

Triple volume is 1.23B × 26 B = **32 GB**, trivially sortable in S3. **The second pass is only affordable because of §3's staging** — without it, a second read costs another 2.7–5.4 h from HuggingFace instead of ~0.2 h from S3.

Why not a Bloom filter. At fp=1e-4 it is only 2.94 GB and fits the existing container, which is genuinely attractive. But false positives **discard real documents** — 123K documents ≈ 0.10B tokens at that rate — and it cannot be made deterministic as a shared mutable structure. Keep it as the fallback if the pre-pass proves operationally awkward, and record the 0.01% loss explicitly if so.

The 8-byte truncation is safe. Birthday collision probability over 1.23B documents at 64 bits is **4.08%** — but that is the probability that *any* collision exists at all; the expected number of colliding pairs is **0.04**, i.e. expected data loss under one document. The current 128-bit width is over-provisioned by 64 bits.

5.4 What exact hashing cannot do, and why that is mostly fine

Near-duplicates survive: boilerplate-differing pages, the same article on two domains, a document differing by one interior whitespace character. Every major 2026 corpus uses MinHash/LSH instead.

Two pieces of counter-evidence say not to reach for it reflexively. FineWeb measured that **global cross-dump MinHash was actively harmful** — the removed data outperformed the kept data. And the often-cited −11.8 MMLU from DCLM Table 19 is a **deduplication** result, not a decontamination one. Meanwhile our upstream sources have each already run their own dedup, so our remaining job is **cross-source only** — a much smaller claim than "we need MinHash."

Recommendation: exact, sharded, global. Do not add MinHash for this build. Record the decision so it can be revisited with a measurement rather than a reflex.

5.5 Ordering is an unrecorded decision

`dedup_and_decontaminate` keeps the **first** occurrence, and bundles run in registry order — so which source "wins" a cross-source duplicate is decided by alphabetical accident. Make it explicit: an ordered source-priority list, highest-quality-source-wins, recorded in the plan.

5.6 The receipt does not record what was removed

`run_bundle` returns `filter_stats.as_dict()` and `_cmd_run` prints it, but `Receipt` **has no filter block** — grep for `duplicates` or `contaminated` in `corpus_receipt.py` returns zero hits. The real duplicate and contamination rates for our own sources exist only in CloudWatch logs from the 2026-08-05 run. Cheap to fix, and until it is fixed no quantitative dedup claim is auditable.

---

6. Decontamination

6.1 What the index holds, verified item by item

`DecontaminationIndex` runs two independent tests, OR'd: the exact content hash, or `minimum_hits=2` distinct 13-gram hits. The shipped index is **149,777 exact hashes + 3,097,372 13-grams**, ~250 MB resident — budget it on Batch, it is not free.

Coverage, MEASURED against the manifest's own `source_counts` (127 keys, 148,458 base-unique texts):

benchmark	present	splits	items
MMLU	✅	all 57 subjects × {validation, test}	62,102
GSM8K	✅	`main/test` only	1,319
ARC-Challenge	✅	test + val	4,687 + 1,194
ARC-Easy	✅	test + val	9,496 + 2,281
HellaSwag	✅	`val` only (test labels are unreleased)	40,145

All four benchmarks we steer on are present, plus seven bonus families. Good.

6.2 ⚠️ But the index is built from 5-shot RENDERED prompts, which kills half of it

The single highest value-per-dollar fix in this review. `eval_bundle.py:141–170`: each MMLU entry is a subject preamble + **five worked demonstration Q/A pairs** + the target question + **one answer choice**.

Consequence 1 — the 149,777 exact hashes are effectively dead for MMLU, ARC and HellaSwag. An exact hash fires only on a corpus document byte-identical to a full OLMES render with one specific choice appended. Real contamination is a bare question, or a question with its options laid out differently. It never looks like that render.

GSM8K is the exception, and it explains the evidence. Its entries are `question + "\n" + answer` — a shape a web page can genuinely match. Which is exactly why the one real-index test that passes is the GSM8K one: "*40/40 GSM8K test questions caught.*" There is no MMLU equivalent, and now we know why.

Consequence 2 — the n-gram half survives, but diluted 21×. VERIFIED: 3,097,372 distinct 13-grams over 148,458 unique texts is **20.9 per text**, against ~438 windows a 400–500 word render would naively yield. The collapse is the set de-duplicating the shared preamble and five demonstrations across ~1,090 items per subject — so what survives as distinct entries are the windows **spanning the target question and its choice**. The question does get indexed, at ~21 windows rather than ~438.

Consequence 3 — the short-item hole is on the index side too. A benchmark question under ~14 words yields windows only in combination with template words (`Question:`, `Answer:`, the previous demonstration's tail), so it can match only a document that reproduces the template. This is worse than the usual framing, which notes only that short *documents* yield zero windows.

FIX: rebuild the index from raw benchmark fields — question alone, question + each choice, question + correct answer — **in addition to** the rendered form. One CPU job, no GPU, using the pinned `ai2-olmo` checkout. DERIVED cost: a bare ~30-word MMLU question yields ~18 windows × 62,102 items ≈ **1.1M new n-grams, about +36% index size and +18 MB**. Cheap, and it restores the exact-hash half.

6.3 Three coverage gaps worth naming

- **MATH, HumanEval, MBPP, DROP, TriviaQA, BBH, MMLU-Pro and GPQA are entirely undefended.** The index covers the 9 OLMo-ladder families only. This mix carries substantial math and code, so if the program ever reports MATH or HumanEval, those numbers have **no contamination defense at all** — while the build prints a healthy-looking `DECON index 149,777 exact + 3,097,372 ngrams` either way. **This is the exact failure mode that matters: an index missing a benchmark we steer on reports success.** Decide the eval suite, then extend the index to match it.
- **GSM8K `train` is absent** (test only). Leakage of GSM8K *train* is the documented failure mode — Nemotron STEM-SFT was seeded from GSM8K/MATH/AOPS train splits. It does not directly inflate a test score, so severity is low, but it becomes real if train items are ever used as few-shot demonstrations.
- **MMLU `dev` is present only as the embedded 5-shot preamble**, not as items. The docstring claiming `{dev, validation, test}` coverage is imprecise and should be corrected.

6.4 The rephrasing hole, which nothing here closes

FinePhrase is a rephrasing of FineWeb-Edu, and **n-gram decontamination is defeated by rephrasing by construction**. If a benchmark item leaked into FineWeb-Edu, its FinePhrase rewrite carries the same knowledge with zero shared 13-grams. Our own measurement of the general case: 13-gram detection scores **F1 0.926 on verbatim text and 0.000 on rephrased text.**

There is no affordable defense. Embedding- or model-based detection over 1.23B documents is not proportionate here. **The honest position is to accept it and disclose it in `limitations[]`** — and to note that HellaSwag is where contamination persists longest while measuring 0.00% n-gram-dirty, which makes it simultaneously our least-served and least-instrumented metric.

6.5 The tier-1 exclusions

Nobody earns a decontamination TRUST verdict across the 17 audited corpora. Exclude 9 items at source — **under 1% of tokens, carrying nearly all of the risk.** The concrete one already identified: `data_provenance_initiative` ships `fc-cot-cot_gsm8k` (GSM8K in Flan CoT format) at 6 repeats with a ~9× cooldown upweight, and costs **0.51% of tokens** to drop.

---

7. Tokenization

7.1 The contract, which holds whatever tokenizer implements it

Four properties the build depends on. A replacement tokenizer must preserve all four.

1. `add_special_tokens=False`**, and we append EOS = 100257 ourselves.** The library default is `True`, and what it then adds depends on a post-processor nobody in this repo owns. Appending it ourselves makes `1 / mean_doc_tokens` a number the packer can assert — which is the entire basis of the EOS gate. **For dolma2 this is moot in the best way:** `post_processor` **is** `null`, so `add_special_tokens` is a no-op in either direction (MEASURED — read the live `tokenizer.json`).
2. **Every id asserted into** `[0, vocab_size)` **before the uint32 cast.** The cast cannot fail: MEASURED on numpy 2.4.4, assigning `np.array([-1, 5], dtype=int64)` into a `<u4` buffer yields `[4294967295, 5]`, and `2**33` yields `0` — both silently. The assertion is load-bearing.
3. `vocab_size` **includes added tokens.** dolma2 is 100,278 real / 100,352 padded, with 22 added tokens at ids 100256–100277. Because `eos_token_id` 100257 lives inside that range, **a tokenizer shipping an empty** `added_tokens` **yields** `eos_token_id: None`**, and Gate A's EOS check then skips silently** — the live defect in task #12, present in two already-published corpora.
4. **EOS is appended at tokenize time, not pack time.** So re-packing at a different `seq_len` requires **re-tokenizing**. A deliberate trade: the EOS is the only document boundary this corpus will ever have, so it belongs inside the unit whose contract guarantees it.

7.2 The tokenizer stays dolma2

No alternative clears a measured downstream gain. Two facts make this more than inertia: `allenai/dolma3-tokenizer` **is** dolma2 (AI2 says so in their own code), and AI2's pre-tokenized shards are **byte-identical to our** `.u32le.bin` — verified by range-read, no NumPy header, valid ids from byte 0. Their 5.93T corpus is therefore a byte copy for us, not a re-tokenization.

7.3 gigatoken — audited, and safer than expected

`github.com/marcelroed/gigatoken`, MIT, Rust with Python bindings, CPU-only. Claims ~1000× over HuggingFace `tokenizers`. Since our corpus is content-addressed and a wrong-but-in-range token id is invisible to every gate we own, **exact output parity was the only question that mattered**; speed was secondary.

What the audit established, all MEASURED by reading source:

- **The pretokenizer regex is byte-identical.** dolma2's `Split` pattern and gigatoken's `OLMO3_REF_REGEX` in `src/pretokenize/fast/olmo3.rs:261` are **the same 115 characters**. The internal name "olmo3" covers OLMo 2 and 3, and dolma2 is exactly that scheme — the README's "OLMo 2/3" row is not a lookalike.
- **The repo's own parity fixture is semantically our tokenizer.** It tests `allenai/Olmo-3-1025-7B`, whose `tokenizer.json` differs from `allenai/dolma2-tokenizer` **only in merges serialization** — same 100,278-entry vocab dict, same 100,000 merges, same pretokenizer, same 22 added tokens, same EOS id. So every existing ID-parity assertion already covers our tokenizer's behaviour.
- **There is no lossy fast mode.** The README's "compatibility vs native" split is an **API-shape** distinction, not a lossy-vs-exact one: `_hf_compat.py` calls the same native Rust backend and then builds transformers-shaped Python objects. Both tiers produce ids from one encoder.

- **We are not forced onto the slow wrapper.** The native `encode_batch` accepts `list[str] | list[bytes]` directly and fans out over rayon — which is exactly the shape our documents arrive in. `encode_batch_list` returns `list[list[int]]` assembled in Rust, avoiding an `awkward` dependency.
- **An unrecognized pretokenizer is a hard error, not a silent wrong split.** Scheme selection is an exact string match with `None` converted to `Err`. This was the scariest hypothesis in the brief and the source refutes it.
- **The legacy-merges risk was raised and then closed by measurement.** `allenai/dolma2-tokenizer` ships the old space-joined form (`'Ġ Ġ'`) that the tested fixture does not. `hf.rs:113–137` accepts both via a `serde(untagged)` enum **over the whole list**, so a partial parse is not representable — either every entry deserializes or the load fails. Verified on our actual file: all 100,000 entries are legacy-form, **0** have a space count other than 1, and for all of them both halves *and* their concatenation resolve in the vocab. The legacy parse reconstructs the merge table exactly.
- **The differential suite is real, and unusually good** — 8 tokenizers including olmo3, 36 adversarial texts (emoji, CJK, RTL, non-NFC decomposed, `"a"*500`), 28 special-token texts including the exact dolma2 added-token set plus a lookalike battery that must *not* match, and `tests/test_encode_dclm.py` runs exact ID parity on **DCLM-baseline documents selected for tokenizer-hostile content** — literally our largest source. **But CI does not run it.**

The speed question, calibrated against our own measurement rather than the README. Our build measures **10.5 M tok/s** with HF `encode_batch` across 32 vCPU (MEASURED), so 1.0T tokens is **26.5 h**. The audit projects **20–60× for our workload**, discounting the README 4× for heterogeneous data, no AVX-512, and Python overhead — putting tokenization at **0.5–1.5 h**.

But tokenization is not the bottleneck, and that is the crux:

stage	today	with gigatoken
read ~6.1 TB of source	~13.5–27 h	**unchanged**
tokenize 1.0T	26.5 h	~0.5–1.5 h
total, serial	~40–53 h	~14–28 h
total, overlapped	~26.5 h (tokenize-bound)	~13.5–27 h (**download**-bound)

So in an overlapped design the win is **0 to ~13 h**, depending on a number we have not measured: our sustained in-region S3 read bandwidth. Our own memory records `publish()` pulling at 0.8 MiB/s *out of region*, which is why in-region rates need measuring rather than assuming.

The real argument for it is not wall-clock — it is retry cost. A 26.5 h tokenize needs `attemptDurationSeconds` far past the default and is expensive to re-run after a bug. A 1 h tokenize is cheap to re-run, which changes how freely the mixture can be iterated. Given this project's history of finding corpus defects *after* publishing, cheap re-tokenization has option value beyond this build.

7.4 Three findings that force a gate rather than a straight adoption

- **CI runs no tests.** `.github/workflows/CI.yml` is stock maturin — it builds wheels and publishes to PyPI on tag. No `pytest`, no `cargo test`. The good differential suite above is **developer discipline, not enforcement**, which means *a version number is not evidence of parity*.
- **Unicode-version divergence, declared WONTFIX.** gigatoken resolves `\p{L}` / `\p{N}` through ICU4X 2.2 (Unicode 17); HuggingFace `tokenizers` 0.20.x uses Oniguruma (Unicode 14). The maintainer closed the

report as WONTFIX. So the two can disagree on recently-assigned codepoints **by design**.

- **A confessed prior bug in exactly our risk shape.** `fast/mod.rs:129–155` documents a fixed defect where pretokens over 65 KB of invalid UTF-8 split **nondeterministically** between code paths. And a related issue's symptom was *same token count, different order* — invisible to every check we own.
- **No fuzzing exists** (zero hits for fuzz/proptest/quickcheck). Testing is differential over fixed corpora: real and three-level, but not generative.

7.5 Recommendation

Do not put gigatoken on the critical path for this build. Keep HF `tokenizers` — it built the 251.2B corpus and every test exercises it. The reasoning is not that the risk is large; it is that **the benefit is currently unmeasured while the risk is not zero**, and tokenization is not the bottleneck.

Measure first, and it is one cheap job: our sustained **in-region S3 read bandwidth**. That single number decides whether tokenization is on the critical path at all. Our own memory records `publish()` pulling at 0.8 MiB/s *out of region*, so in-region rates must be measured, not assumed.

If the gate is worth running, this is it — specific enough to submit without a follow-up question:

> Point the repo's own `tests/tokenizers/test_hf_parity.py` fixture at `allenai/dolma2-tokenizer` instead of `allenai/Olmo-3-1025-7B`, and run it together with `tests/test_encode_dclm.py` and `test_owt_matches_hf` at `OWT_MAX_BYTES=0` (the full ~12 GB). **Pass = exact `np.array_equal` on ids for every document, plus `get_vocab_size()` reporting 100,278.** Redirecting the fixture is the required step, not an optional one: it is the only thing that actually **executes** the legacy space-joined merges path, which §7.3's proof only argued for statically. On FarmShare or AWS, never locally.

What makes the residual risk acceptable rather than merely small: tokenizer parity is **run-once-then-frozen**. The corpus is immutable, so one job's evidence covers its entire life.

The honest limit of that gate, stated because it undercuts the recommendation: the Unicode trigger set is a few thousand codepoints, so a one-million-document gate can come back clean while a run a thousand times larger still diverges on a few thousand documents. A clean gate proves a **low rate, not zero**. The only complete check is the full 26.5 h HF run — which destroys the reason to adopt in the first place.

It flips for the next build. There, re-tokenization is the *point* rather than a cost: the ~$600 SuperBPE A/B needs multiple passes over the same text, and the download is already paid for by the staged copy in §3.

Two supporting reasons this ordering is right regardless: the corpus is content-addressed, so a divergence found after publishing means a full re-copy rather than a patch; and swapping the tokenizer changes nothing about the blockers in §0, which is where the risk actually lives.

7.6 ⚠️ The tokenizer implementation we already use is unpinned

Found while checking the above, and it matters more than the gigatoken question.

`tokenizers` **appears nowhere in** `pyproject.toml`**.** Declared dependencies are exactly `boto3` and `numpy<2.5`, while `corpus_build.py:631` does `from tokenizers import Tokenizer` and `tokenize_documents` calls `encode_batch` on it. So **the package that decides every token id in the corpus is resolved at container-build time to whatever PyPI served that morning.**

This is precisely the hazard that file's own `numpy<2.5` comment exists to prevent — and it applies with more force here, because a numpy mismatch crashes while a tokenizer change **silently emits different ids that stay in range and decode**. The Unicode-table axis above makes it concrete rather than theoretical: two builds of the same corpus on different mornings can differ on documents containing recently-assigned codepoints, and no gate we own can tell.

This environment has 0.22.2, so we are not on the flagged 0.20.x generation locally — but production is unconstrained.

Fix before the first ingest job: declare and pin `tokenizers`, and record the resolved version in the receipt beside `wheel_version`, so a corpus can name the implementation that produced it and not merely the vocabulary it used.

---

8. Validation and publish topology

8.1 Gate A costs 6 network round trips per object, and 5 of them are serial

MEASURED-IN-CODE, per manifest entry:

check	call	threaded?
Gate A decision loop	`s3.head` — real size vs declared `bytes` (`validate.py:703`)	**yes**, `--head-workers`
`_observed_size`	`s3.head` , cached per key (`pretrain_tokens_v1.py:222`)	**no**
`check_decode_smoke`	`4 × s3.get_range` at seeded offsets, 16 KB each (`:240`)	**no**
`check_first_bytes_not_npy`	`s3.get_range(key, 0, 8)` — the `\x93NUMPY` sniff (`:438`)	**no**
`check_seq_len_alignment`	cache hit, free	n/a

The 85-minute figure everyone quotes is **MEASURED live** and recorded at `pretrain_tokens_v1.py:205–210`: "*Gate A ran ~85 min at 0.3% CPU and ~15.8 round trips/s — purely latency-bound, which is what pushed the first promotion attempt past its 2 h timeout.*"

That reconciles with the model: 10,049 objects × 6 calls = 60,294 round trips over 85 minutes = 11.8 rt/s, against a recorded 15.8 (the gap is the per-group LIST plus manifest GETs). **Gate A is `objects × 6 × latency`, and the CPU is idle.**

8.2 It does not fit any plausible timeout at either shard size

objects	round trips	serial	with `head_workers=16`
10,049 (**measured**)	60,294	**85 min**	~71 min
20,000 (report's shard size)	120,000	**2.82 h**	~2.35 h
40,000 (**code's shard size**)	240,000	**5.63 h**	~4.7 h

`--head-workers` **alone barely helps, and the reason is Amdahl's law:** it threads exactly one of six calls, so even at infinite head workers the five serial calls still cost 83% of the original time. The CLI help is accurate but narrowly scoped — it describes *the decision loop*, while five of the six calls live in the profile checks that run afterwards.

⚠️ **The "1-hour validate cap" is UNVERIFIED and should not be planned against as fact.** What the repo actually records: `edullm-validator:7` was registered at `attemptDurationSeconds = 7200` (2 h), revisions 1–6 had `timeout: null`, and the live revision is **12 with its timeout recorded nowhere**. The 1-hour figures in `docs/PLATFORM-INTEGRATION.md` are `attemptDurationSeconds: 3600` on **platform GPU/smoke** job definitions, not the dataset validator. **Get the live number before submitting:** `aws batch describe-job-definitions --job-definition-name edullm-validator`. Either way, 1 h fails and 2 h fails; the honest requirement at default settings is **≥ 4 h**.

Promotion, by contrast, is already solved. It is ~2 round trips per object but **is** threaded on both phases, so 20,000 objects at 16 workers is ~10–15 min.

8.3 The fix, in order of leverage

1. **Raise `max_pool_connections`.** `s3.py:196` builds `boto3.client("s3")` with **no `Config` at all**, so it inherits botocore's default of **10**. `validate.py:2477` documents this for another path. **Threading against a 10-connection pool self-throttles**, so this must come first or the next step underdelivers.
2. **Thread the five profile-check calls.** `validate.py:577` already does exactly this pattern for the size sweep; `profiles/pretrain_tokens_v1.py` has **zero** threading. At 16 workers, 40,000 objects drops from 5.63 h to roughly **21 minutes**. This is task #10, and it is the right fix.
3. **Drop the redundant HEAD.** The npy sniff and the decode windows both need the object size, and a ranged GET already returns it in `Content-Range` — so the cached HEAD can go, taking 6 calls to 5. Smaller win, but it is pure subtraction.

Once this is fixed, shard size stops being forced by the validator and can be chosen on mixture error and OLMo-core's read pattern — which is the correct basis. Note mixture error at the code's current 25,001,984-token shard is **0.007%–0.278%** across the stage-1 sources, an order of magnitude better than the 0.33% the report cites, so it does not constrain the choice either.

8.4 Publish as TWO datasets, not one

`build_mixture` is scoped to exactly **one group of one dataset**, enforced in three places. And `PATH_LABEL_KEYS = ("source", "domain")` is exactly two levels deep, with `labels_from_path` raising on anything deeper — so "**stage**" **cannot be a third path segment.** Three options, and the recommendation is unambiguous:

option	verdict
one dataset, stage fused into `source` (`dclm-s1`, `dclm-s2`)	works, but doubles the source vocabulary and makes every per-source predicate awkward
one dataset, trainer stages it	defers a decision the corpus should record
two datasets	**recommended**

Why two datasets is right, and it is not just tidiness:

- **A cooldown is sequential by definition**, so nothing ever needs one mixture spanning both stages — which is the only thing the single-dataset constraint forbids.
- **Each validates separately.** Stage 2 at 4,000 objects is **0.56 h serial — it fits today, with no code change.** That turns one impossible validate into one easy one plus one that needs the §8.3 fix.
- **Ordinal blast radius is contained per stage.** The §0 renaming hazard stops crossing between them.

- **Stage 2 can be rebuilt or reweighted without touching stage 1** — and stage 2 is where the QA and reasoning shares are least certain, so it is exactly the half most likely to be rebuilt.

8.5 Interleaving is trainer-side. Definitively.

Task #15 wants micro-batches that are not domain-pure. **It cannot be done in the data**, and here is the proof: `labels` is a field of `ManifestEntry` — **one dict per shard path** — derived from the path segments, and Gate A recomputes it and compares by **full dict equality**. An interleaved shard holds documents from N sources but can carry only one `source` label, so it would be either mislabelled or rejected.

So the fix belongs in the trainer's data loader, and the setting is already identified: `MoELoadBalancingLossGranularity`, which defaults to `local_batch` in four places. Worth **0.13–0.18 perplexity and +5–6 GSM8K**, and it **cannot be annealed in** — switching at 10% of training recovers only ~55%. With only 2 shared experts of 6 active, routing quality is load-bearing rather than incidental.

---

9. The job plan

Nothing here auto-publishes. Every AWS job goes through the platform submission form and needs a human release.

Phase 0 — code and measurement, no AWS, ~2 days

All of it is pure code or a read-only query, and **all of it must land before the plan is frozen**, because several items change the plan.

#	item	why it is here	effort
1	**Wire the FinePhrase id partition** into `_reader_for`	Changes the plan. Cannot be retrofitted after tokenization	~5 lines + the budget correction below
2	**Correct** `_CHARS_PER_TOKEN` 6.0 → ~4.6	Halves the read	1 line
3	**File-shard val bundles** via the existing `_shard_slice`	Removes the other half of the read amplification	~30 lines
4	**Replace the dedup set with a flat** `np.uint64` **pre-pass**	The current design OOMs at 1T	~1 day
5	**Pin** `tokenizers` in `pyproject.toml`	Production currently resolves whatever PyPI serves	1 line
6	**Thread the profile checks + raise** `max_pool_connections`	Gate A does not fit otherwise	~20 lines
7	**Record** `FilterStats` **in the receipt**	Without it no dedup claim is auditable	~10 lines
8	**Fix the boundary-marker prefix guard**, or comment why the table must stay at one entry	A future addition to it is silently a no-op today	~5 lines
9	**Drop** `data_provenance_initiative`	Ships GSM8K in CoT format; costs 0.51% of tokens	registry edit
10	**Query the live validator timeout**	`edullm-validator:12`'s timeout is recorded nowhere	one read-only call

⚠️ **Item 1 has a trap worth naming:** applying the partition without dividing the reader's character budget by the keep fraction means every bundle finishes and *then* fails `verify` on unfilled shard refs. The two changes ship together or not at all.

Phase 0b — three measurements that gate specific sources

measurement	what it decides	cost
In-region S3 read bandwidth	Whether tokenization is on the critical path at all; decides the gigatoken question	one cheap job
Nemotron-CC-Math's real dolma2 token count	Its 133B is CARD with no tokenizer named. Also **gated — a human must accept the license** before its text column can even be named	one authenticated footer read (~700 KB)
Dolma 3 adult-content prevalence	Blocking for that source. Sample at **random offsets** — a prior attempt could not separate the signal from HuggingFace preview ordering	~1 h

Also worth the hour: **5 of 9 stage-2 sources have no measured mean document length**, and the AI2 dolma3 QA source (GPT-4o-mini-rewritten multiple choice) is the one plausibly near the 20-token EOS floor. A random-offset sample settles it.

Phase 1 — freeze the plan, then stage

1. **Freeze the mix.** Every source, both stages, final shares. This is the real gate.
2. **Generate the complete plan** for all sources of both stages. Record its `plan_id`.
3. **Stage sources to** `s3://edullm-landing/_src/` — ~4.21 TB, ~$194 for two months.

Phase 2 — ⚠️ a mandatory single-bundle smoke test

`_reader_for` **has never run against live HuggingFace from inside a Batch container, and the code says so** (`corpus_build.py:901–904`). Run one bundle against the smallest source before committing an array job. This is the single cheapest risk reduction available.

Phase 3 — the build, in waves

~100 bundles across 4–6 array waves. Per-bundle resume already works: `bundle_is_done` re-HEADs and compares sizes, and re-running a lost bundle is byte-identical (verified — nine bundles reproduced identical digests).

stage	tokens	shards @25.0M	Gate A serial	Gate A threaded
stage 1	~900B	~36,000	5.07 h ❌	~19 min
stage 2	~100B	~4,000	0.56 h ✅	~2 min

Phase 4 — publish two datasets

`pretrain/final-stage1-900b` and `pretrain/final-stage2-100b`. Per stage: `verify --deep`, publish, Gate A, promote. **Remember writing a** `manifest.json` **fires EventBridge and promotes automatically** — to stage without promoting, cancel the validator job or disable the rule first.

---

10. What is still unverified

Listed because the distinction between measured and inherited is the only thing that makes the rest trustworthy.

Blocking a specific source:

- **Nemotron-CC-Math's text and id columns are UNVERIFIED** — the dataset is **gated**, so its schema cannot be read without authentication. It is the only stage-1 source whose text column we cannot name. Its 133B is CARD with no tokenizer named, and `3plus` **is not a loadable config** (it is the union of `3` and `4plus`), so an ingest row naming it fails to resolve.
- **Dolma 3's adult-content prevalence** is unmeasured and blocking.
- **The dolma3 midtraining mix is** `allenai/dolma3_dolmino_mix-100B-1125` and ships `.jsonl.zst` **text**, not pre-tokenized shards — so re-tokenization is required, filtering is possible, and **zstd is unreadable by** `corpus_read` **today**. Its 323 directory names *are* category labels, so QA is selectable by prefix with no classifier. Note `allenai/dolma3-dolmino-mix-1025` does not resolve.

Structural unknowns:

- **Three sources have no usable document id** — both full-DCLM repos and Cosmopedia. Every `sha256(id) % N` partition and anti-join in this plan depends on one. Decide the surrogate before ingest; a hash-of-text id is not stable across a re-download.
- **252B of FineWeb-Edu breaks the FinePhrase anti-join.** That volume must come from `sample-350BT`, which is FinePhrase's exact parent, so ~72% of the rephrase source lands in the corpus as real text too.
- **FinePhrase ships** `finish_reason` **and the plan ignores it.** `max_tokens=2048` with a measured p99 rewrite length of 1,847–1,948 tokens, so ~0.5–1.5% of faq/tutorial rewrites end mid-sentence — concentrated in the highest-token-mass documents. Nemotron-CC-Math, Cosmopedia and Nemotron-Specialized have the same exposure with **no such field at all**.
- **Per-source normalization is inconsistent and invisible to every validator.** `finepdfs-edu` already ran FTFY, so it is NFC-normalized, ligature-split and straight-quoted; no other source is. And **peS2o is a PDF-extraction source in disguise** — Grobid over PDF, with OCR errors "filtered not fixed" and no FTFY.
- **Cosmopedia's leading space, MEASURED at 303/303 documents across all 8 configs.** A Mixtral SentencePiece `▁` detokenization artifact. Under byte-level BPE the space-prefixed and bare forms of a word are **different token ids** (verified against the real dolma2 vocab: `The` = 791, `ĠThe` = 578), so every document's first token is its mid-sentence variant, immediately after our appended EOS. One `lstrip()` fixes it — **but it must not be applied globally, because leading whitespace is semantic in code.**
- **Never unescape** `\n`**.** Real Nemotron bytes contain `\neq` and `\nabla`; the obvious heuristic would destroy exactly the math the source is being bought for.
- **Gate A's cost at ~40,000 objects** is extrapolated from a measured 85 min at 10,049. Never tested.
- **MegaMath-Web's post-filter pool size** is unmeasured.
- `c7i` **pricing** is quoted from memory, not a live price API.

The largest residual risk, unchanged: every mixture study behind the shares in `FINAL-DATASET-REPORT.md` was run on a **dense** model. **No mixture ablation on a sparse MoE exists at any scale.** Transfer to our architecture is unverified, and that unknown is larger than any measurement error in this document.

A methodological caution that applies to several numbers above: the Cosmopedia and Nemotron measurements come from HuggingFace `/first-rows`, which is a head read. A prior wave found head sampling on content-clustered parquet wrong by **10×** once already. What makes the Cosmopedia result convincing is the **100% rate across 8 independent configs**, not the sample size.

One free improvement worth taking: a per-shard **encoding receipt** — a handful of O(1) predicates bolted onto the tokenize path (does any document start with whitespace, does any contain `<|endoftext|>`, is any text non-NFC). It converts eight of the UNVERIFIED rows above into MEASURED ones as a side effect of a build that was going to read the bytes anyway.